

The EWM Lattice and Its Worlds

HLLSets, Emerging Lattices, and Multi-World Representation

Companion note to *Emerging World Models* (August 2026)

Alex Mylnikov

Alexandr Mylnikov Jr.

Deependra Kumar

September 29, 2026

Abstract

This companion note elaborates the object at the center of the Emerging World Models (EWM) framework: the **HLLSet lattice** and the **worlds** it carries. The important word in EWM is *emerging*. The lattice is never built whole; it is grown as measurements arrive. An HLLSet is a compact bit-vector “sketch” of a token collection, and the HLLSet algebra (*one universe*: every sketch is a node of one Boolean lattice; *the morphism is the contract*: a deterministic, immutable, content-addressed map from tokens to nodes) is what makes the growth safe. Many users—many **worlds**—can write into the same lattice simultaneously, and the algebra makes it impossible for them to build anything wrong: the join is monotone and convergent, so independent contributions coexist rather than conflict. The hash function (its *type* and its *seed*) is a parameter of the measurement, not part of the lattice, so different users hash into different worlds that are *virtually isolated even when they share HLLSets*: a shared sketch is the same node on the representation plane, but each world reads it through its own LUT. This built-in isolation is a free consequence of the algebra, not a bolt-on policy; we call it the *emergence security* of HLLSet Algebra. We formalize worlds as BSS-thresholded node sets with interpretations, prove the closure and isolation properties, describe the internal sub-pipeline by which two worlds communicate (materialize in world 1, transcribe, ingest in world 2), and close by identifying the bridge with the Grothendieck-topology target of the main paper. The lattice’s size— 2^{32K} nodes at the default configuration—is treated in a single paragraph as the variety a compact presentation affords.

1 Introduction

The main EWM paper [1] establishes four foundation principles and grounds them in HLLSet Algebra [2]. Its Section 5 (“HLLSet Algebra: Concrete Instantiation”) treats the HLLSet bitmask as a holographic plate of 32,768 patches, and its Section 7 (“Toward Grothendieck Topology”) anticipates geometric morphisms as universal bridges. The present note is the missing middle: it takes the object that the **ewm-state-machine** project actually materializes—the **HLLSet lattice**, and the **worlds** that coexist on it—and develops the structural consequences that the framework has been using informally.

The word that carries most of the weight in *Emerging World Models* is **emerging**. We never build a whole dense HLLSet lattice; there is no reason to, and at the default configuration there could be no hope of it. The lattice is built *as we go*, node by node, as measurements arrive. What makes this safe at scale is HLLSet Algebra, which gives three things:

1. **One universe.** Every HLLSet is a node of one Boolean lattice (order $\subseteq$, join $\cup$, meet $\cap$, ring Δ). Nothing is created that was not already a node; a computation *discovers* a node by naming a lattice expression. Building the lattice is discovering nodes, never restructuring it.
2. **The morphism is the contract.** The token $\rightarrow$ node connection is a chosen measurement morphism; only its IICA properties—Idempotence, Immutability, Content Addressability—

are required, and any IICA algorithm is interchangeable. The bit vector is anonymous: it does not remember which token or which scheme set it.

3. **Monotone, convergent accumulation.** The join is union, and union is idempotent, commutative, and associative. Independent contributions therefore accumulate without coordination, without consensus, and without conflict.

The consequence is the theme of this note: **many users—many worlds—can write into the same lattice simultaneously, and the algebra makes it impossible for them to build anything wrong.** A node that one user adds is a node the lattice now has; another user adding overlapping nodes cannot corrupt it, because union is monotone and content-addressed. There is no lock, no registry, and no validation pass to get wrong.

The hash function (its *type* and its *seed*) enters as a parameter of the measurement, not as part of the lattice. That single choice partitions the users into **virtually isolated worlds**: two users who hash differently generally place the same token at different nodes, and they read any shared node through their own LUTs. Even when they *share* HLLSets, each world’s interpretation of those sketches is its own. This is a built-in isolation that HLLSet Algebra gives us *for free*—the *emergence security* of the representation. We make it precise in Sections 3–4.

2 HLLSets and the Emerging Lattice

2.1 What an HLLSet is

An **HLLSet** is a compact sketch of a token collection. Instead of storing the tokens, it stores a fixed-length bit vector whose set bits are the *measurement sites* the tokens hashed into. It is the object the **ewm-state-machine** project manipulates: a token stream goes in, one sketch comes out, and the sketch is what all later computation reads.

The geometry of the sketch is *soldered*:

- precision P fixes the **register count** $M = 2^P$;
- each register occupies b bits of the flattened vector;
- a token t is hashed once and decomposed into a *register index* and a *trailing-zero state*, which together name a single **bit address** in the vector.

The decomposition is the insertion rule

$$\text{reg}(t) = h(t) \bmod M, \quad \text{tz}(t) = \text{clamp}(\text{trailing_zeros}(h(t) \gg P), 0, b - 1), \quad (1)$$

and the address is the flat pair

$$\text{bit}(t) = \text{reg}(t) \cdot b + \text{tz}(t) \in \{0, 1, \dots, M \cdot b - 1\}. \quad (2)$$

Definition 2.1 (HLLSet). *Fix a token space $\mathcal{T}$. The **bit plane** is $\mathbb{B} = \{0, \dots, M \cdot b - 1\}$, and an **HLLSet** is a subset $H \subseteq \mathbb{B}$, represented by its indicator vector—a flattened bit vector of length*

$$M \cdot b = 2^P \cdot b. \quad (3)$$

The sketch of a token collection $A \subseteq \mathcal{T}$ is

$$H(A) = \bigcup_{t \in A} \{\text{bit}(t)\}, \quad (4)$$

so a token collection is represented by the set of sites it marks, not by the tokens themselves. With the defaults $P = 10$ and $b = 32$ the vector is 32,768 bits long—a 32K-bit presentation.

2.2 The HLLSet lattice

The one fact that turns sketches into a representation is that subsets of a common plane are automatically a lattice: inclusion orders them, union joins them, intersection meets them.

Definition 2.2 (The HLLSet lattice). *Let $\mathbb{B} = \{0, \dots, M \cdot b - 1\}$ be the bit plane. The **HLLSet lattice** (the EWM lattice) is the Boolean lattice*

$$\mathcal{B} = (2^{\mathbb{B}}, \subseteq, \cup, \cap, \Delta), \quad (5)$$

with order $\subseteq$ (bitwise subset), join $\cup$ (bitwise OR), meet $\cap$ (bitwise AND), and ring addition Δ (symmetric difference). An HLLSet is a node of $\mathcal{B}$ —one 32K-bit presentation—and every computation in the state machine—unions, R-links $A \cap B$, the DRN decomposition, ring residuals, basis frames—names a node and discovers it rather than creating it.

Three of the algebra’s properties are what the rest of the note relies on, and they are worth stating once, plainly:

1. **One universe.** Every HLLSet is a node of the same $\mathcal{B}$; nothing is created that was not already a node. A computation *discovers* a node by naming a lattice expression, and two paths that name the same expression reach the same node.
2. **The morphism is the contract (IICA).** The token $\rightarrow$ node connection is a chosen measurement morphism $\mu : \{\text{token collections}\} \rightarrow \{\text{HLLSets}\}$; the only requirement is that μ be Idempotent, Immutable, and Content-Addressed. The bit vector forgets provenance by design, and any IICA algorithm is interchangeable.
3. **Monotone, convergent accumulation.** The join is $\cup$, which is idempotent, commutative, and associative. Independent contributions therefore accumulate without coordination, consensus, or conflict.

Remark 2.3. The bit-plane lattice is exactly the “one universe” of the project’s `BOOLRING.md`: every HLLSet is a node of the same Boolean lattice, and every morphism result already exists in it. Definition 2.2 is the paper-level statement of that fact.

2.3 Emerging: the lattice is grown, never built whole

We never materialize a dense HLLSet lattice, and there is no reason to try. At the default configuration it has 2^{32K} nodes, of which only a vanishing fraction will ever be needed. The lattice is **emerging**: it is grown node by node as measurements arrive.

Concretely, a site subscribes to the tokens it cares about, hashes each one through μ , and unions the resulting site into the sketches it holds. The set of nodes in existence is the set of expressions the participants have actually named—the committed sketches, their unions and intersections, their DRN components, their ring residuals. Nothing else exists, and nothing else needs to. This is what makes the representation scale: the cost is proportional to the measurements taken, not to the size of the plane they could occupy.

The monotonicity of $\cup$ is what makes the growth safe, and it is why *many users can write into the same lattice at once*. Suppose users $u_1, \dots, u_m$ independently add their sketches. Whatever order the additions arrive in, and however they interleave, the accumulated node is the union of their contributions; union is associative, commutative, and idempotent, so the final node is independent of interleaving and re-addition. A user cannot overwrite another’s node, cannot leave the lattice in an inconsistent state, and cannot even tell (from the lattice alone) that anyone else is writing. The algebra makes it impossible to build anything wrong.

Principle 2.1 (Emergence safety). *A shared HLLSet lattice can be written concurrently by any number of independent participants. Every contribution is a monotone join, and joins commute, associate, and absorb repetition; hence the accumulated state is well-defined and order-independent, and no participant can corrupt another’s contribution. There is no lock, no registry, and no consensus protocol to implement.*

Remark 2.4 (Variety from a compact presentation). It costs one paragraph to say how much room this gives us. The flattened vector has $M \cdot b = 32\text{K}$ bit positions, so the lattice has $2^{M \cdot b} = 2^{32\text{K}} = 2^{32768}$ nodes at the defaults ($P = 10$, $b = 32$); that is a 9,865-digit number, about 2^{32502} times the count of atoms in the observable universe. The point is not the number itself—we will never visit more than a sliver of it—but that a presentation of 32K bits (4 KB, when packed) can name so vast a variety of distinct states. That is the room in which the worlds of the next section emerge.

3 The Hash Function as an Index of Worlds

3.1 The hash configuration

The insertion rule (1) depends on a hash function. In the project it is MurmurHash3 x64-128 (lower 64 bits) [3], and the realization exposes two independent choices:

- the **hash type** (which algorithm);
- the **seed** s , already present in the interface as `murmur3_hash_seeded(data, seed)`.

We call the pair (h, s) a **hash configuration**—it is the h -part of a world, as formalized next. A bootstrap scheme (e.g. n -gram windows versus n -seed catalogs, with the channel seeds $\text{seed}(n, \text{dim}) = (\text{dim} - 1) \cdot 3 + (n - 1)$) is a further, encoding-level choice; it enters the same framework as a discrete family of seeds, and we return to it in Section 3.4.

Definition 3.1 (World). A **world** is a triple $w = (h, s, \tau)$:

- a hash configuration (h, s) (hash type and seed), which induces the **measurement morphism**

$$\mu_w : 2^{\mathcal{T}} \longrightarrow 2^{\mathbb{B}}, \quad \mu_w(A) = \bigcup_{t \in A} \{\text{bit}_{h,s}(t)\}, \quad (6)$$

with $\text{bit}_{h,s}$ as in (2); the **world image** $\text{Im}(w) = \mu_w(2^{\mathcal{T}})$ is the set of atoms it can produce, and the **world lattice**

$$\mathcal{B}_w = \text{join-closure of } \text{Im}(w) \subseteq \mathcal{B} \quad (7)$$

is its set of reachable nodes (every finite union of world atoms);

- a BSS threshold τ . Writing

$$\text{BSS}_w(X) = \frac{|X \cap B_w|}{|B_w|} \quad (8)$$

for the BSS of a node X against the world's basis B_w (the ring over that world's HLLSets), the **thresholded node set**

$$\mathcal{N}_\tau(w) = \{ X \in \mathcal{B}_w : \text{BSS}_w(X) \geq \tau \} \quad (9)$$

is the collection of nodes that belong to the world, together with the relations between them that pass τ ;

- an **interpretation**

$$\iota_w : 2^{\mathbb{B}} \longrightarrow 2^{\mathcal{T}}, \quad \iota_w(X) = \bigcup_{b \in X} \mathcal{L}_w[b], \quad (10)$$

the materialization map that reads a node's active bits back as the tokens they stand for under (h, s) , through the reverse index $\mathcal{L}_w$ of that hash configuration (one fiber per bit address).

The thresholded node set is the world's extent; the interpretation is the world's reading of it. Two worlds are **the same world** when both their thresholded node sets and their interpretations agree on the nodes they share; they are **distinct worlds** otherwise.

In the state machine, a world is therefore what is left after the BSS threshold is applied—the surviving nodes and relations—together with the LUT that gives those nodes their meaning. The

threshold is what makes the collection a *world* rather than a bare sublattice; the interpretation is what makes it this world rather than another.

Everything in the model—tokens, token hashes, and HLLSets—is *IICA*: Idempotent, Imutable, and Content-Addressed. All checking therefore happens *at the creation of a new element*, never later. Ingest performs its whole pass at once: it registers every ingested token in the token LUT, computes each token’s (reg, tz) position, rehashes each token under the other hash functions, and at the end of the batch returns the SHA-1 identifier of the HLLSet it submitted to the `hllsetLUT`. An HLLSet produced by a set operation likewise receives its identifier when it is registered. The rule is therefore simple and absolute:

Principle 3.1 (Existence is registration). *An HLLSet exists only after it has been registered in the `hllsetLUT`. An HLLSet outside the `hllsetLUT` does not exist—even if a program holds its bits. To bring it to life, register it first. Every HLLSet algebra operation computes and returns a content-addressed ID, and an ID is only ever born at registration.*

Remark 3.2. This is what makes registration the natural home of every consistency check: the collision test against the existing identifiers, and the collision-tolerant bookkeeping that follows from it (Section 3.5), are executed once, at the moment the element comes into being. Nothing is deferred, so no later reader has to repair a half-built object—IICA guarantees there is nothing to repair.

Proposition 3.3 (Idempotent registration, eventual consistency). *Registration is idempotent: registering the same HLLSet again leaves the `hllsetLUT` unchanged. Consequently concurrency is not an issue—if two participants register the same element, or one registers it twice, the result is the same LUT—and a set of LUTs written concurrently converges by re-registration alone. No lock, ordering, or deduplication pass is needed for the registrations to agree.*

Proof. A register operation is keyed by content: it inserts the pair (identifier, HLLSet) into the LUT. Registering (k, H) when (k, H) is already present is a no-op, and the touch count is monotone (a CRDT join, pointwise max), so $R \circ R = R$ on the same element. For two concurrent registrations of the same element the two writes produce the same entry, and any later registration of that element is absorbed; hence the LUT state after a finite quiescent period is independent of the interleaving. This is the same monotonicity as Principle 2.1, now at the registry rather than the lattice level. $\square$

Remark 3.4 (Why re-registration converges the LUTs). The guarantee does not need the writes to be ordered or synchronized, only that registration is idempotent and keyed by content. A participant that is unsure whether an element made it into a LUT can simply register it again: if it is present the second registration changes nothing, and if it is absent the second registration completes it. So the LUTs are *eventually consistent by re-registration*¹—the same content key always names the same element, and re-asserting a key is always safe.

Proposition 3.5 (Same hash type, different seeds). *Let $w = (h, s)$ and $w' = (h, s')$ use the same hash type with different seeds. On the common token domain, the two measurements are related by a bijection of atoms $\phi : \text{Im}(w) \rightarrow \text{Im}(w')$, and ϕ extends uniquely to a lattice isomorphism $\hat{\phi} : \mathcal{B}_w \rightarrow \mathcal{B}_{w'}$ that commutes with $\cup, \cap, \Delta$.*

Proof. Both worlds realize the same abstract map $\text{token} \rightarrow (\text{register}, \text{tz}) \rightarrow \text{bit}$; only the two hash values differ. On each token t , set $\phi(\text{bit}_{h,s}(t)) = \text{bit}_{h,s'}(t)$. Distinct tokens can give the same atom (collisions), so ϕ is well defined as a map of atoms exactly when collisions agree; where they do, the induced map on the generated Boolean algebras is the unique lattice homomorphism extending ϕ , and it is a bijection because both worlds generate free Boolean algebras on the

¹This is the registry-level analogue of the lattice’s union invariant: both are monotone, content-keyed accumulations, so both converge without coordination.

same token domain modulo the same collided partition. Boolean operations on a bit plane are coordinatewise, so $\cup, \cap, \Delta$ are carried to $\cup, \cap, \Delta$. $\square$

Changing the hash *type*, not only the seed, likewise changes the realized atom map, not the algebra. This is the precise sense in which EWM is hash-agnostic.

Theorem 3.6 (Closure of a world’s representation under computation). *Let $W_1, \dots, W_k \in \mathcal{B}_w$. Let*

$$E = E(W_1, \dots, W_k)$$

be any expression built from the operations $\cup, \cap, \setminus, \Delta$ and the constants $\emptyset, \mathbb{B}$. Then $E \in \mathcal{B}_w$. In particular, the DRN components $D = A \setminus B$, $R = A \cap B$, $N = B \setminus A$, the ring residual, and every basis-frame projection of a world’s HLLSets remain in that world.

Proof. $\mathcal{B}_w$ is by construction the Boolean subalgebra of $\mathcal{B}$ generated by the atoms $\text{Im}(w)$: it is closed under finite union and complement, hence under the derived operations $\cap, \setminus, \Delta$ and contains $\emptyset, \mathbb{B}$. Each W_i is a finite union of atoms of $\text{Im}(w)$; any expression E is therefore a finite Boolean combination of those atoms, which lies in the generated subalgebra. Induction on the expression E gives the claim. $\square$

The theorem is a statement about the *representation* plane. The interpretation layer is not closed in this way— ι_w need not commute with $\cup, \cap$ beyond the fiber union it is defined by—and that asymmetry is exactly what makes worlds separate: computing inside a world stays in its representation, while *reading* it requires that world’s LUT. Shared representation, separate interpretation.

3.2 Interpretation and the separation of worlds

The representation plane is uniform across worlds: there is one ambient lattice $\mathcal{B}$, and every world’s nodes are nodes of it. A world cannot therefore be separated from another by *which nodes exist*—they all live in the same 2^{32K} -node lattice. What distinguishes worlds is how the nodes are *read*. The formal objects of Definition 3.1 split accordingly: the thresholded node set $\mathcal{N}_\tau(w)$ is set-theoretic, and the interpretation ι_w is the reading applied to it.

Proposition 3.7 (Representation-level sharing, interpretation-level separation). *Two worlds may share the very same HLLSet node X yet be distinct worlds. They are distinct exactly when the interpretations differ on a shared node,*

$$w \neq w' \iff \exists X \in \mathcal{N}_\tau(w) \cap \mathcal{N}_\tau(w') : \iota_w(X) \neq \iota_{w'}(X), \quad (11)$$

and their interpretations may still overlap, $\iota_w(X) \cap \iota_{w'}(X) \neq \emptyset$, when the two token sets share tokens while differing elsewhere.

Proof. Let X be a node reachable under both configurations (a shared HLLSet, e.g. a constant node or a node of common structure). Definition 3.1 assigns to X the token set $\iota_w(X)$ under w and $\iota_{w'}(X)$ under w' . If the two assignments differ, X is the same bit vector in both worlds but two different readings; this is separation with shared representation. The converse is the definition of distinctness. Overlap of the two readings is possible because the fibers $\mathcal{L}_w[b]$ and $\mathcal{L}_{w'}[b]$ are different sets of tokens that may intersect; nothing forces disjointness. $\square$

Proposition 3.8 (Hash configurations give distinct worlds). *Suppose the token domain is rich enough that two different hash configurations $w = (h, s)$ and $w' = (h', s')$ separate on some token, i.e. $\text{bit}_{h,s}(t) \neq \text{bit}_{h',s'}(t)$ for some $t \in \mathcal{T}$. Then w and w' are distinct worlds in the sense of Definition 3.1.*

Proof. The two configurations place t at different bit addresses. Hence the fiber containing t under $\mathcal{L}_w$ is indexed by an address different from the one under $\mathcal{L}_{w'}$; materializing a node that contains the address of t under w returns t (among others), while the corresponding reading under w' reads a different fiber. So the interpretations differ on that node. $\square$

The decisive point is that the bit vector itself is semantically inert: nothing in the HLLSet remembers that it was produced under h rather than h' . All meaning lives in the LUT, and the LUT is a different object for every hash configuration. Worlds are separated by their LUTs, not by their bit vectors.

Corollary 3.9 (No cross-talk, restated). *Inferring world w' from world w cannot be done by the lattice operations alone: by Proposition 3.7 a node read under w' differs from the same node read under w exactly at the interpretation step, which the lattice does not perform. The two worlds meet only in the shared representation, where they are indistinguishable until materialized; to go from one to the other one must materialize and re-ingest—the bridge of Section 4.*

This is the technical content of the phrase “all these worlds stay fully isolated in this EWM lattice”. Isolation is not enforced by a lock, a namespace, or a validation pass; it is a consequence of the interpretation layer being a *module over the representation*: every world has the same nodes, and each world reads them through its own LUT.

Principle 3.2 (Virtual isolation as built-in security). *Two users who hash differently inhabit **virtually isolated worlds** on one shared lattice. At the representation level they genuinely share nodes—the sketch of a common token collection is the same bit vector for both—but at the interpretation level ι_w recovers tokens through the LUT of (h, s) . A user who does not hold the other’s hash configuration and LUT cannot read the content their node stands for, however much of the lattice they inspect. The isolation is not a policy applied on top of the data; it is a property of the algebra, and it is therefore obtained for free.*

Remark 3.10 (What “for free” means here). The representation is whole and shared: we do not partition, encrypt, or gate the bit plane. *Virtual* isolation means that the separation is carried by the interpretation, so it costs one hash configuration and one LUT per world, not a division of the lattice. It also means the separation is *structural rather than adversarial-tamper-proof*: it prevents *reading* another world’s content and prevents *confusing* one world’s nodes with another’s, but it is not a substitute for cryptography where an adversary can choose the encodings. Its value is that a representation built for one purpose cannot silently leak into, or be corrupted by, a representation built for another.

3.3 The lattice as a holographic presentation of many worlds

Write Ω for the space of hash configurations $w = (h, s)$, and let $\mathcal{B}_\Omega = \bigcup_{w \in \Omega} \mathcal{B}_w$ be the union of all reachable nodes across the family. Because the ambient lattice is a single object, the collection of worlds is not a collection of separate memories; it is a family of *readings* of one memory.

Principle 3.3 (Holographic multi-worlds). *Let Ω be any finite or countable family of hash configurations. Then:*

1. **Common representation.** *Every world $w \in \Omega$ is read through the same lattice $\mathcal{B}$; the representation is not duplicated per world. All worlds have access to the full 2^{32K} nodes.*
2. **Separate interpretation.** *What makes two worlds distinct is their interpretation map ι_w (Definition 3.1), not their nodes: they may share a node exactly and still differ (Proposition 3.7).*
3. **Joint closure of the representation.** *Every expression over nodes drawn from a single world stays in that world’s subalgebra (Theorem 3.6); the thresholded node set $\mathcal{N}_\tau(w)$ is the world’s actual extent.*

4. **Global representability.** The whole of $\mathcal{B}_\Omega$ is a subset of one 2^{32K} -node lattice; the cost of hosting $|\Omega|$ worlds is a per-world LUT and threshold, not a multiplication of the representation.

The word *holographic* is used in the same sense as in the main paper: the lattice top plus its local coordinates reconstructs each world’s state, and a world is a *view* of the whole. Formally, the structure over Ω is not the disjoint union of the representations—those all coincide, being the same $\mathcal{B}$ —but the fibred set

$$\mathcal{E}(\Omega) = \bigsqcup_{w \in \Omega} (\mathcal{N}_\tau(w), \iota_w) \quad (12)$$

whose fibre at w is that world’s thresholded node set together with its interpretation. The shared representation supplies the base geometry; the fibre supplies the reading. Reading a world is choosing a fibre; reading all worlds at once is reading the total space $\mathcal{E}(\Omega)$ through the one common $\mathcal{B}$.

Remark 3.11 (Why the worlds do not multiply the cost). If the worlds had separate representations, the state-bit budget would be $|\Omega| \cdot M \cdot b$. In the holographic arrangement the representation budget stays $M \cdot b$ bits *shared*; each world adds only its LUT and its threshold. Since the cache stores a bounded working set (the ring, the tip, the committed turns), the practical overhead is the per-world LUT, not the lattice.

3.4 Adding bootstrap schemes to the family

The bootstrap scheme is a second, encoding-level parameter and fits the same picture. An n -gram over a stream and an n -seed catalog over the same tokens use different channel seeds,

$$s(n, \dim) = (\dim - 1) \cdot 3 + (n - 1), \quad s(1, \cdot) = 0, \quad (13)$$

and therefore are different worlds $(h, s(n, \dim), \tau)$ in the sense of Definition 3.1. The scheme lives on the *LUT name* (**ng**: versus **ns**:), which is the materialization coordinate of the world; the HLLSet itself stays scheme-agnostic, exactly as the project’s `SEPARATION.md` insists. Thus the hash-configuration part of the world family is

$$\Omega = \{ w = (h, s) : h \in \text{hash types}, s \in \text{seeds} \cup \{s(n, \dim)\}_{n, \dim} \}, \quad (14)$$

a discrete parameter space whose points are the worlds (each with its own BSS threshold and interpretation ι_w). Precision and register width, by contrast, are not points of Ω ; they size the lattice itself (Section 2) and are fixed before the world family is considered.

3.5 Content addressing and the multi-hash extension

A world’s nodes are immutable and content-addressed, so each carries a key. In the realization, the default key is a SHA-1 digest of the HLLSet itself. One SHA-1 key, however, has only 2^{160} possible values, while the lattice has $2^{32K} = 2^{32768}$ nodes. A single digest therefore cannot name the lattice injectively, and as the number of stored HLLSets grows the key space becomes a hard ceiling. The gap is not marginal.

Theorem 3.12 (Single-hash capacity bound). *No function whose output is a single b -bit hash is injective from the HLLSet lattice $\mathcal{B}$ to its keyspace when $b < 32K$. In particular, for b -bit SHA-1 ($b = 160$) across N stored HLLSets the addresses collide when N approaches $2^{b/2} = 2^{80}$, and no key assignment with one digest can cover more than 2^{160} of the 2^{32768} nodes.*

Proof. A function into 2^b values has image of size at most 2^b , so it cannot be injective on a domain of size 2^{32K} once $b < 32K$; this is the pigeonhole principle. For a random b -bit hash the birthday bound puts the first collision at $N \sim 2^{b/2} = 2^{80}$; SHA-1 has $b = 160$. $\square$

3.5.1 The concrete design: one HLLSet, three hashes, three LUTs

The fix reuses the materializer’s own pattern. There, the 1-gram and the 0-seed are the *same* hash-seed pair (the shared channel), and the other channels are reached either by a different hash value (3-gram) or by a different seed (3-seed). HLLSet identity is extended the same way, but on the HLLSet itself: we take the HLLSet, hash it once for the default identifier and rehash the same HLLSet under two further independent hashes.

$$\kappa_3(H) = (\text{SHA1}(H), H_1(H, s_1), H_2(H, s_2)), \quad (15)$$

where H is the HLLSet, H_1 may be another 160-bit function or a wider one such as Murmur-256, and $s_1 \neq s_2$ are independent seeds. The three digests are three names for the *same* object; no token is involved, and nothing about the sketch changes.

The vocabulary matters here. A LUT holds *collections*: a token LUT holds tokens (or token hashes), and an `hllsetLUT` holds HLLSets. The words *bucket* and *register* are reserved for the internal structure of an HLLSet and are not used for lookups. The correspondence is exact: every token is kept in three token LUTs, and every HLLSet is kept in three `hllsetLUT`s, under three different IDs, while the object is the same. (This is the same move as the n -gram LUTs, where an x -gram is stored under its own key and points at its first token; here the HLLSet is stored under three keys and all three point at the same object.)

So the state machine keeps three `hllsetLUT`s, one per digest,

$$\text{sha1-hllsetLUT}, \quad \text{hash1-hllsetLUT}, \quad \text{hash2-hllsetLUT},$$

each mapping a digest to the collection of HLLSets registered under it,

$$L_{\text{sha1}}, L_{h1}, L_{h2} : \text{digest} \longrightarrow \text{collection of HLLSets}. \quad (16)$$

This mirrors the token side, where the same token appears in three LUTs, and the n -gram side, where the same window is indexed under its own key. It is the project’s existing “one LUT per channel” bookkeeping, now applied per hash rather than per token channel.

The second and third LUTs grow lazily. By Principle 3.1 the check happens once, when the HLLSet is registered, so it is there that collisions are detected. A SHA-1 collision is close to impossible at any realistic scale, so the extra LUTs are not populated eagerly. An HLLSet H is added to the second LUT *only if its SHA-1 identifier collides* with an already-registered HLLSet, and to the third LUT *only if the second also collides*. In the overwhelmingly common case an HLLSet lives in the first LUT alone; the second LUT is a de-collider consulted only under a SHA-1 collision, and the third is a de-collider consulted only under a collision in the second. The two extra LUTs therefore hold at most a negligible fraction of the collection, and the memory cost of the whole scheme is one key per HLLSet plus a rare second or third key.

Definition 3.13 (Three-stage retrieval). *Retrieval descends the LUTs only as far as it must:*

1. (**stage 1, the common case**) look up the SHA-1 LUT at a . If the collection $L_{\text{sha1}}(a)$ holds exactly one HLLSet, return it—no further hashes are computed;
2. (**stage 2**) otherwise compute the second hash b and look up the second LUT. If $L_{h1}(b)$ holds exactly one HLLSet, return it;
3. (**stage 3, essentially never**) otherwise compute the third hash c and intersect the three collections,

$$L_{\text{sha1}}(a) \cap L_{h1}(b) \cap L_{h2}(c) = \{H\}. \quad (17)$$

Only the first hash is paid on the common path; the second is paid only when the SHA-1 collection is contested, and the third only when the second is contested too.

Proposition 3.14 (Retrieval is exact). *Let H be stored under the lazy regime above, with digests $\text{SHA1}(H) = a$ and, when computed, $H_1(H) = b$, $H_2(H) = c$. Then*

1. **Completeness.** At every stage where H 's digest is looked up, H is present: at stage 1 by registration; at stage 2 because $L_{sha1}(a)$ was contested, which triggered H 's registration in the second LUT; at stage 3 because $L_{h1}(b)$ was contested, which triggered its registration in the third. Hence retrieval always returns the target.
2. **Uniqueness.** Any $H' \neq H$ returned by stage 3 lies in all three collections and therefore collides with H on all three hashes, SHA1, H_1 and H_2 simultaneously. For independent b -bit hashes this costs a factor 2^{-b} per extra coordinate—about 2^{-320} for the two extra—and requires the rare two-stage collision to have occurred at all.

Proof. (1) By Principle 3.1, H exists because it was registered, and registration places it in the second LUT exactly when a SHA-1 collision makes stage 2 reachable, and in the third exactly when a second-LUT collision makes stage 3 reachable. Hence at each stage the entry needed for that stage exists, and retrieval always returns the target. (2) Membership in the stage-3 intersection means $SHA1(H') = SHA1(H)$ and $H_1(H') = H_1(H)$ and $H_2(H') = H_2(H)$; for independent b -bit hashes each coincidence costs a factor 2^{-b} , giving 2^{-160} per extra coordinate. $\square$

Remark 3.15 (Capacity, not adversarial strength). The three coordinates together name up to 2^{480} values, well past the 2^{80} single-key collision point, and the two extra LUTs cost only the two additional digests per HLLSet. Two caveats are worth stating. The construction assumes the three hashes are *independent*; a single function re-seeded still shares that function's structural weaknesses. And the purpose is *capacity and collision resolution*, not adversarial collision resistance: SHA-1 is long broken for adversarial purposes, so a system that must resist a computing adversary should use a modern hash family while keeping the same three-coordinate retrieval.

3.5.2 The general extension

If more than three coordinates are ever needed, the same construction extends to an m -fold extendable key,

$$\kappa_m(H) = (\text{SHA1}(H), H_1(H), \dots, H_{m-1}(H)), \quad (18)$$

with one LUT per coordinate, all indexed by the same H . The address space is 2^{bm} instead of 2^b , and the collision condition becomes conjunctive.

Proposition 3.16 (Multi-hash extension). *Let κ_m be an m -fold independent b -bit key as in (18). Then:*

1. **Coverage.** κ_m can address up to 2^{bm} distinct HLLSets. Full coverage of the lattice needs

$$m \geq \left\lceil \frac{32K}{b} \right\rceil, \quad \text{i.e. } m \geq 205 \text{ for SHA-1 } (b = 160), \quad (19)$$

or $m \geq 128$ with SHA-256 ($b = 256$).

2. **Collision law.** $\kappa_m(H) = \kappa_m(H')$ with $H \neq H'$ requires a simultaneous collision in all m coordinates; its probability falls as 2^{-b} per additional coordinate, giving $\Pr[\text{collision}] \approx N^2/2^{bm+1}$ for N stored HLLSets.
3. **Practical choice.** Full-coverage integrity needs $m = 205$, but the concrete three-coordinate design of (15) is chosen for the actual scale: its intersection collides with probability below 2^{-300} for $N \leq 2^{100}$, and the second coordinate alone already reduces the SHA-1 collection by 2^{-160} .
4. **Cost.** The key grows by b bits per coordinate—20 bytes per SHA-1 coordinate—so the concrete $m = 3$ key adds 40 bytes, and even the full-coverage $m = 205$ key is about 4.1 KB, comparable to the HLLSet presentation it names.

Proof. (1) The image of κ_m lies in $(2^b)^m = 2^{bm}$ values, so injectivity requires $bm \geq 32K$; the ceiling follows. (2) A collision of the tuples is the intersection of m independent events, each of probability $\approx N^2/2^{b+1}$; independence multiplies them, giving $N^2/2^{bm+1}$. (3) Substituting $N = 2^{100}$ and $m = 3$ gives $2^{200}/2^{481}$; $N = 2^{80}$ and $m = 2$ gives $2^{160}/2^{321}$. (4) Each coordinate contributes b bits, $b/8$ bytes. $\square$

Remark 3.17 (The multi-seed sketch as an injective address). There is a second, representation-side route to the same end. Ingesting a token collection under m independent seeds produces m HLLSets in m bit planes; the m -tuple of those bit vectors is itself the address, and the map from a single-seed lattice to the multi-seed product,

$$H \mapsto (H_0, \dots, H_{m-1}), \quad H_s = \text{sketch of } H \text{ under seed } s, \quad (20)$$

is injective once the coordinates are independent—it is the same extension seen from the representation rather than the key side. This is exactly the multi-seed ingestion the materializer already runs ($\text{SEEDS} = [0, 1, 2]$ and the channel seeds $s(n, \text{dim})$): the 0-seed is the shared default, and the additional seeds are the extra coordinates, one catalog per seed.

Remark 3.18 (What is and is not assumed). The extension assumes the m coordinates are *independent* hash functions. Reusing one SHA-1 with a seed is a practical way to generate them, but then the coordinates share SHA-1’s structural weaknesses; the correct reading is that the key is as strong as the weakest independent function used. The point of Proposition 3.16 is capacity, not adversarial collision resistance: SHA-1 is long broken for adversarial purposes, and a system that needs the latter should use a modern hash family while keeping the same m -fold extension. The registry cost is the same either way.

Remark 3.19 (Where the LUTs keep it). The `ewm-state-machine` already keeps touch-count `HllsetLuts`, merged as a monotonic CRDT, one per token channel. The three-hash design adds two more `hllsetLUTs` of the same kind, one per hash, and each LUT entry is a *collection* of HLLSets (a digest may be carried by several of them), as LUTs hold collections by definition. Because the second and third LUTs are grown lazily, they add essentially nothing to the steady-state cost—only the rare collision populates them—while the per-entry touch counts are kept exactly as in the single-LUT case. The extension is bookkeeping plus collision-tolerant LUT collections, not a new data structure.

4 Cross-World Communication

4.1 The internal sub-pipeline

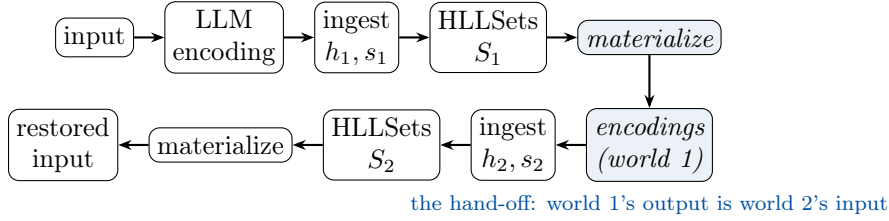
Two worlds cannot exchange lattice nodes directly—by Corollary 3.9 a world’s nodes simply do not name the other world’s nodes. Communication must pass through the only interface the worlds share: the **encodings**. The bridge from world w_1 to world w_2 is the composition

$$\Phi_{w_1 \rightarrow w_2} = \underbrace{\text{ingest}_{w_2}}_{\text{hash } h_2, s_2} \circ \underbrace{\text{transcribe}}_{\text{LUT fiber} \rightarrow \text{value}} \circ \underbrace{\text{materialize}_{w_1}}_{\text{bits} \rightarrow \text{LUT fiber}}, \quad (21)$$

where `materialize` is the LUT-first inverse read of μ_{w_1} (every candidate of every active bit is kept), and `transcribe` is the protocol-level mapping that turns a recovered encoding into a value world w_2 can hash. In diagram form:

$$\begin{array}{ccccc} \mathcal{B}_{w_1} & \xrightarrow{\text{materialize}} & \mathcal{L}_{w_1} & \xrightarrow{\text{transcribe}} & \text{encodings} \\ & & & & \downarrow \text{ingest, } h_2, s_2 \\ \mathcal{B}_{w_2} & \xleftarrow{\text{measure}} & \text{Im}(w_2) & \xleftarrow{\mu_{w_2}} & \text{encodings} \end{array} \quad (22)$$

In the project’s terms this is exactly the side-car loop run twice, with the materialized output of the first pass used as the input stream of the second. The two worlds share the same surrounding pipeline—input, LLM encoding, materialization, restored input—and meet only at the encoding layer:



Theorem 4.1 (Bridge laws). *Let $\Phi = \Phi_{w_1 \rightarrow w_2}$ be the bridge (21), and assume materialization is exact on the tokens that built the source nodes (the standing guarantee of LUT-first materialization: every builder is returned). Then:*

1. **Determinism (IICA).** Φ is deterministic and content-addressed: the same source state yields the same target node, and the target is named by its SHA-1 content key.
2. **Idempotence on builders.** Ingesting the same bridge transcript again yields the same target node; hence $\Phi^2 = \Phi$ on states whose materialization is exact.
3. **Order preservation.** Φ is monotone: $A \subseteq A'$ in $\mathcal{B}_{w_1}$ implies $\Phi(A) \subseteq \Phi(A')$ in $\mathcal{B}_{w_2}$.
4. **Strict coarsening in general.** If materialization returns collateral candidates at collided bits, transcribing the larger transcript can only add atoms to the target; Φ is then a monotone inflation, not a bijection.

Proof. (1) Materialize, transcribe, and ingest are each deterministic functions of their inputs; composition preserves determinism and content addressability (the IICA composition theorem of the main paper). (2) If materialization recovers exactly the builders B of the source node, the transcript is B ; ingesting B under (h_2, s_2) is $\mu_{w_2}(B)$, which is unchanged on a second pass with the same transcript. (3) Materialization returns the builders of A ; if $A \subseteq A'$ then the builder set of A is contained in that of A' (up to collisions), and μ_{w_2} is a union over the transcript, hence monotone. (4) Collisions at materialization add candidates; ingest unions their atoms, so the target can only grow. $\square$

4.2 What crosses the bridge, and what does not

Principle 4.1 (The bridge carries encodings, not content). *The bridge $\Phi_{w_1 \rightarrow w_2}$ transmits the encoded form of the state—token identifiers, codec tokens, n -gram windows, grid cells, or whatever the world’s front-end emits—and never the source content those encodings stand for. The receiving world re-embeds the encoding under its own hash configuration; it does not inherit the sender’s semantics.*

This is not a defect but the architecture. The lattice is vocabulary-agnostic and semantics-agnostic; it knows bits and their LUT fibers, not the meaning of the tokens. Each side that participates in a bridge is responsible for its own encoder and decoder. The bridge’s guarantee is structural: *the same encodings, placed by the receiver’s measurement, with the receiver’s collisions handled by its own LUT.* The fidelity of a bridge is therefore the fidelity of the two encoders, not of the lattice; the lattice’s contribution is that it never loses a builder and never confuses two worlds.

Remark 4.2 (Round-trips). The composite $\Phi_{w_2 \rightarrow w_1} \circ \Phi_{w_1 \rightarrow w_2}$ is not the identity in general: materializing extras from collisions, and re-encoding, can coarse-grain the state. Exactness holds on the builders (Theorem 4.1(2)), which is the sense in which the pipeline “restores the input”: it restores the *encodings* exactly on everything that built the lattice state, and at most adds hash-collision neighbours.

This note	ewm-state-machine
precision P	<code>hllset_contracts::P = 10</code>
register count $M = 2^P$	<code>M = 1 << P = 1024</code>
bits per register b	<code>BITS_PER_REG = 32</code>
bit plane size $M \cdot b$	<code>TOTAL_BITS = 32768</code>
node of $\mathcal{B}$	<code>hllset_core::HLLSet</code> (a Roaring bitmap)
bit address	<code>hllset_contracts::BitAddress</code>
bit = $\text{reg} \cdot b + \text{tz}$	<code>BitAddress::from_reg_tz</code>
$\mu_w, w = (h, s)$	<code>murmur3_hash_seeded / BitAddress::of_token_seeded</code>
$\text{seed}(n, \text{dim})$	<code>conv(n, dim); SEEDS = [0,1,2]</code>
$\text{Im}(w)$	the content keys of the committed HLLSet nodes
$\text{BSS}_w(X) = X \cap B_w / B_w $	the BSS / similarity-coverage readouts
$\mathcal{N}_\tau(w)$	the nodes surviving the world’s BSS threshold
ι_w (interpretation)	LUT-first inverse read (<code>ewm-scene</code> , <code>LutIndex</code>)
content key of a node (default)	<code>h:<sha1></code> (SHA-1 of the HLLSet)
additional hashes (two)	$H_1(H, s_1), H_2(H, s_2)$ (eq. (15))
three LUTs	<code>sha1-hllsetLUT</code> , <code>hash1-hllsetLUT</code> , <code>hash2-hllsetLUT</code> (digest $\rightarrow$ collection)
retrieval	SHA-1 LUT singleton, else the three-way intersection (Def. 3.13)
extendable key κ_m	an m -tuple of independent digests (Proposition 3.16)
LUT type	<code>hllset_morphisms::HllsetLut</code> $((\text{name}, \text{key}) \rightarrow \text{TH}, \text{CRDT})$
materialize_w	the interpretation applied to a node
ingest_w	<code>hllset_morphisms::Ingest</code> (+ <code>ns:G1..G3</code> LUTs)
$\Phi_{w_1 \rightarrow w_2}$	the two-pass side-car loop (notebooks 02–21)
$\mathcal{E}(\Omega)$	the <code>ewm-git</code> commit DAG over one shared store

Table 1: Correspondence between the lattice/world formalism and the artifacts that realize it.

4.3 Communities of worlds

The bridge composes, so a family of worlds connected by bridges forms a directed graph whose vertices are hash configurations and whose edges are bridges. Two consequences deserve to be stated because they are what the multi-world picture buys:

- **Migration.** Information can be moved from a world with hash type h_1 to a world with hash type h_2 without re-architecting the lattice: only the two hash configurations and the transcription protocol must be agreed upon.
- **Adapters as bridges.** A new LLM, vision encoder, or decision model entering the system is a new vertex; its adapter is the transcription protocol on the incoming and outgoing edges. The `ewm-state-machine` project’s repeated finding—*the only model-specific code is a tiny adapter*—is the engineering face of Principle 4.1.

5 Correspondence with the Implementation

The definitions above are not idealized; each has a named counterpart in the code. Table 1 collects them.

Two implementation facts are worth calling out.

The world is a configuration, not a copy. Nothing in `hllset-core` or `hllset-contracts` knows about worlds. The seed is a parameter of `BitAddress::of_token_seeded` and `Ingest`; the hash type is a parameter of the hashing module. Reusing one lattice across worlds is therefore already the default behaviour: there is no “world” type because there is nothing to switch—only a hash configuration to vary, and a threshold to set.

Gates are single-world interpretations. The project’s gates (G_u , and the G1-scoped `bss_g1`) project a state onto one dimension or one world’s codebook. In the present language a gate is a restriction of a world’s interpretation ι_w , not of the representation: the one-sided gate (frames do not leak across a gate) is the operational shadow of Corollary 3.9.

6 Relation to the Grothendieck-Topology Target

Section 7 of the main paper proposes Grothendieck topology as the target formalism, with geometric morphisms as universal bridges. The construction of this note supplies that target with its concrete objects:

- the **base site** is the discrete parameter space Ω of hash configurations;
- a **world** $w \in \Omega$ is a point of the base, and its stalk is the pair $(\mathcal{N}_\tau(w), \iota_w)$ —the thresholded node set together with the interpretation, i.e. its fibre of $\mathcal{E}(\Omega)$;
- a **bridge** $\Phi_{w_1 \rightarrow w_2}$ is a morphism of sites: its “direct image” is the re-encoding ingest_{w_2} and its “inverse image” is materialize_{w_1} , the pair being adjoint in the order of Theorem 4.1(3);
- the **pullback consistency** that a geometric morphism requires is the global-fiber invariant of the project’s `SEPARATION.md`: within one world’s LUT, one bit address points to the same fiber, so a materialization read is well defined. Across worlds the fiber is reinterpreted—that is exactly what makes the worlds distinct—and the bridge is the morphism that transports it.

What is still missing—and what we do not claim here—is the covering structure (the analogue of the bootstrap covering families in the main paper) and a genuine sheaf condition on Ω . The bridge is a geometric morphism in the order-theoretic sense; making it a morphism of sheaf toposes is future work. Meanwhile the finite, fully explicit lattice of Sections 2–4 is the “pre-sheaf” stage of that programme, exactly as the main paper describes its own status.

7 Summary

1. An **HLLSet** is a compact sketch of a token collection: a 32K-bit vector at the defaults, marking the measurement sites the tokens hashed into. The **HLLSet lattice** $\mathcal{B} = 2^{\mathbb{B}}$ is the Boolean lattice of all such sketches (order $\subseteq$, join $\cup$, meet $\cap$, ring Δ); every HLLSet is a node of it.
2. The word **emerging** is literal: the lattice is *grown*, never built whole. Participants name the node expressions they need (sketches, unions, intersections, DRN, ring residuals) and nothing else exists. The cost is proportional to the measurements taken, not to the 2^{32K} -node plane they could occupy.
3. Because the join is union—idempotent, commutative, associative—many users can write into the same lattice at once, and the algebra makes it **impossible to build anything wrong** (Principle 2.1). No lock, no registry, no consensus protocol.
4. Tokens, token hashes, and HLLSets are all **IICA**, so every check happens at creation: **an HLLSet exists only after it is registered** in the `hllsetLUT` (Principle 3.1). Set operations return content-addressed IDs, never anonymous objects.
5. Registration is **idempotent**, so concurrency is not an issue: re-registering an element is always safe, and concurrently written LUTs are **eventually consistent by re-registration** alone (Proposition 3.3).
6. The **hash function (type and seed)** is a parameter of the measurement, so the same lattice carries a family of **worlds** $\Omega \ni w = (h, s)$. A world is its *thresholded node set* $\mathcal{N}_\tau(w)$ together with its *interpretation* ι_w .
7. Users who hash differently are **virtually isolated**: they may share the very same HLLSet and still read it differently, because each world recovers tokens through its own LUT (Principle 3.2). Separation lives at the interpretation level, not on the representation plane; it is a property of the algebra, obtained for free.

8. Worlds communicate only through the **bridge** $\Phi_{w_1 \rightarrow w_2} = \text{ingest}_{w_2} \circ \text{transcribe} \circ \text{materialize}_{w_1}$, which is deterministic, idempotent on builders, and monotone, and which carries **encodings**, **not content** (Principle 4.1). The structure over Ω is the **holographic presentation** of the world family: one shared representation with per-world interpretation fibres, and the bridge is the concrete realization of the main paper’s geometric-morphism target.
9. A single SHA-1 key cannot name the lattice (2^{160} against 2^{32768}). The materializer’s multi-hashing move fixes it at the HLLSet level: hash the same HLLSet under SHA-1 plus **two more hashes** (another 160-bit function, or Murmur-256, under two seeds), one **hllsetLUT** per hash. The second and third LUTs are grown **lazily**, only on a collision at the preceding level, so a SHA-1 lookup almost always settles retrieval on its own; escalation intersects the collections (Definition 3.13, Proposition 3.14). The coordinates name up to 2^{480} values; the same construction extends to $m = 205$ for full coverage (eq. (15), Theorem 3.12, Proposition 3.16).

Acknowledgment

This article was written in collaboration between the authors—Alex Mylnikov and Deependra Kumar—and an AI assistant (an instance of the DeepSeek model working within the `ewm-jepa` workspace). The human authors developed the EWM framework, the HLLSet algebra, and the `ewm-state-machine` realization, and directed the line of argument; the AI assistant helped to draft and structure the text, work through the formal statements and proofs, and verify the correspondence with the implementation. The conceptual content is the authors’ own.

References

- [1] Alex Mylnikov, Deependra Kumar, *Emerging World Models: A Structural Framework for Self-Organizing World Representations*, consolidation document, August 2026. (Main paper; companion to this note.)
- [2] *HLLSet Algebra: A Content-Addressed Set Algebra for Emergent Ontology*, ASTES Journal, 2026.
- [3] A. Appleby, *MurmurHash3*, 2011. x64-128 variant; the project uses the lower 64 bits with a configurable seed.
- [4] W. R. Ashby, *Design for a Brain*, Chapman & Hall, 1952; and *An Introduction to Cybernetics*, 1956.
- [5] A. Grothendieck et al., *Théorie des topos et cohomologie étale de schémas*, SGA 4, Lecture Notes in Mathematics 269, Springer, 1972.
- [6] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, *HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm*, AofA, 2007.